LAB5 결과 보고서

STM32_1 인터럽트와 PWM — FPGA 피아노·신호등 제어·ADC

과목 전자전기컴퓨터설계실험 2

실험일 2026년 10월 19일

성명 김태이

학번 2023440158

GitHub github.com/tae2yi/UOS_ece2/tree/4ff988d/lab5

시연 영상 github.com/tae2yi/UOS_ece2/tree/master/lab5/report/final/videos

1 실험 범위와 결과 요약

10월 19일 실험에서 FPGA 교재 실습 3개(피아노, 신호등 제어, ADC)를 Combo II-DLD 보드에서, STM32_1 과제 2개(버튼·TIM6 인터럽트로 LED 간격 변경, 버튼으로 PWM 비율 변경)를 NUCLEO-G474RE에서 동작시키고 영상 5개로 남겼다. 이 보고서는 영상과 영상 속 소리에서 읽은 값을 예비보고서의 예상과 비교한다. 보드 사진은 모두 영상의 장면이다.

실험	영상에서 확인한 것	예상과 비교
피아노	8음 주파수 262.00~523.00 Hz	설계값과 0.11 Hz 이내
신호등	보행 초록 변화 17번, 점멸·노랑·방향 전환	모든 변화가 계산과 같은 프레임
ADC	ch0·ch1 전환, LED 0~1111_1110	전환은 됨. LED8이 한 번도 안 켜짐
과제3	LED 간격 1 → 2 → 4 s	규칙은 일치. 두 번째 누름 뒤 1.47 s 늦음
과제4	듀티 50 → 75 → 25 → 50 → 75 %, 1 kHz	25 % 단위 순환과 주기 1 ms 일치

피아노, 신호등, 과제4는 측정 분해능 안에서 예상과 같았다. ADC는 실습 중 채널 전환이 되지 않아 DIN 출력 시점을 고친 수정본을 썼고 전환은 확인했지만, 최하위 LED가 영상 내내 꺼져 있었다. 수신이 한 비트 늦는 것으로 설명되며 시뮬레이션으로 확인했다(5장). 과제3은 디버그 모드로 실행해 두 번째 누름에서 코어가 멈춘 시간만큼 간격이 길어졌다(6장).

2 실험 환경과 측정 방법

구분	사용한 것
FPGA 보드	Combo II-DLD, Spartan-7 xc7s75fgga484-1, 신호등 모듈(EXT41~64)
MCU 보드	NUCLEO-G474RE (STM32G474RE)
계측기	Tektronix TBS2000B 오실로스코프(과제4)
FPGA 설계	lab5_01_piano, lab5_02_traffic, lab5_03_adc (ADC는 board_fix 수정본)
MCU 코드	Glaysia/ece2_474 v1.1.0 + ece2_474_routine.c (STM32_1_TASK 3, 4)
기록	휴대전화 영상 5개 (30 fps, 음성 포함)

영상에서 값을 읽은 방법

- 피아노: 영상 음성(48 kHz)에서 50 ms마다 여덟 음의 고조파 크기를 비교해 음이 나는 구간을 찾고, 그 구간에서 가장 센 고조파의 주파수를 고조파 차수로 나눴다.
- 신호등·과제3: 프레임마다 LED 위치의 초록 화소 수를 세어 켜짐을 판정하고, 상태가 바뀐 프레임의 시각을 적었다(한 프레임 0.033 s). 손가락이 LED를 가린 프레임은 판정에서 뺐다.
- ADC LED 값과 오실로스코프의 듀티 표시는 장면을 확대해 사람이 읽었다.

측정 코드:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/report/final/make_figures.py

측정값:
https://github.com/tae2yi/UOS_ece2/tree/4ff988d/lab5/report/final/data

3 FPGA-01 피아노

설계와 사전 검증

버튼 8개 중 하나가 눌리면 note_select가 음의 주파수를 내고, square_tone이 50 MHz 클록을 세어 반주기 50,000,000 / (2f) 클록마다 피에조 출력을 뒤집는다. 두 개 이상 누르거나 떼면 무음이다. 실험 전에 매뉴얼 TB와 파형 확인용 TB가 Icarus·Vivado 모두 통과했고, 50 MHz에서 WNS +14.043 ns로 타이밍을 충족했다.

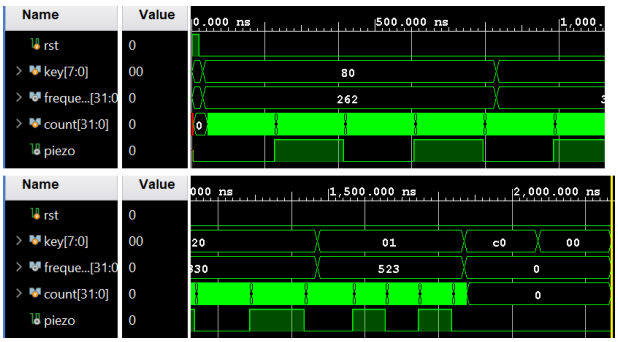


그림 1. FPGA-01 Vivado 파형(tb_view, CLK_HZ=10000, 두 줄 배치). key 80·20·01에서 반주기 19·15·9클록으로 토글, 두 키(c0)와 무입력에서는 piezo=0.

보드 관찰



그림 2. FPGA-01 보드 사진. 입력 조건: 키패드 버튼 8개를 왼쪽부터 차례로 누름. 확인한 동작: 누를 때마다 다른 높이의 소리. 장면 위에 영상 시각과 그 구간 소리에서 측정한 주파수를 적었다.

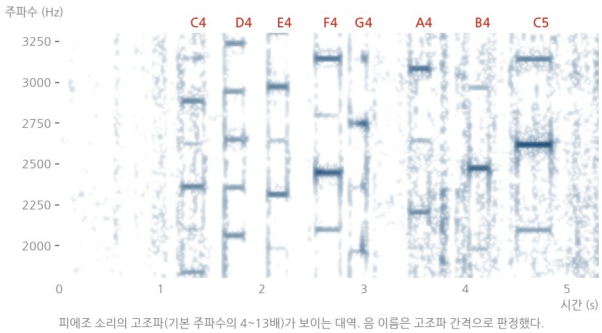


그림 3. FPGA-01 영상 음성의 스펙트로그램(1.8~3.3 kHz). 각 음의 고조파가 가로줄로 보이고, 줄 간격이 기본 주파수다.

음	구간 (s)	설계 (Hz)	측정 (Hz)	차이	n
도(C4)	1.25~1.50	262.002	262.00	+0.00	9
레(D4)	1.70~1.95	294.000	294.02	+0.02	7
미(E4)	2.10~2.35	330.003	330.01	+0.01	13
파(F4)	2.50~2.85	349.001	349.04	+0.04	7
솔(G4)	2.90~3.10	392.003	392.11	+0.11	7
라(A4)	3.50~3.70	440.001	439.97	-0.03	7
시(B4)	4.05~4.30	494.003	493.94	-0.06	5
높은 도(C5)	4.55~4.95	523.002	523.00	+0.00	5

측정값:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/report/final/data/piano_notes.csv

설계 열은 반주기 클록 수를 정수로 자른 뒤의 주파수(50 MHz / (2 × 반주기))이고, n은 측정에 쓴 고조파 차수다. 피에조는 낮은 기본 주파수보다 2~3 kHz 근처에서 소리를 잘 내므로, 기본파 대신 그 대역의 n배 고조파를 재고 n으로 나눴다. 그림 3에서 도(C4)는 2358 Hz(9배)와 2882 Hz(11배), 높은 도(C5)는 2615 Hz(5배)가 뚜렷하다.

예상과의 비교

여덟 음이 버튼 순서대로 한 번씩 났고 측정 주파수는 모두 설계값과 0.11 Hz 이내였다. 구간이 0.2~0.4 s로 짧아 고조파 하나의 주파수 분해능이 수 Hz이고 이를 n으로 나누므로, 이 차이는 측정 분해능 수준이다. 음 사이에는 무음 구간이 있어 버튼을 떼면 소리가 멈추는 것도 확인된다. 두 버튼 동시 누름과 리셋은 이번 영상에 없다.

4 FPGA-02 신호등 제어

설계와 사전 검증

second_tick이 1초에 한 클록짜리 tick과 매초 앞 0.5초 동안 HIGH인 blink를 만든다[2]. traffic_sequence는 tick마다 phase를 올려 0~10초 초록(4~10초는 보행 초록 점멸), 11~12초 노랑을 거친 뒤 다음 방향으로 넘어간다. flicker가 HIGH인 동안은 방향·phase를 0에 묶고 노랑과 보행 빨강을 blink로 점멸한다. 매뉴얼 TB와 파형 TB가 통과했고 WNS +15.148 ns였다.

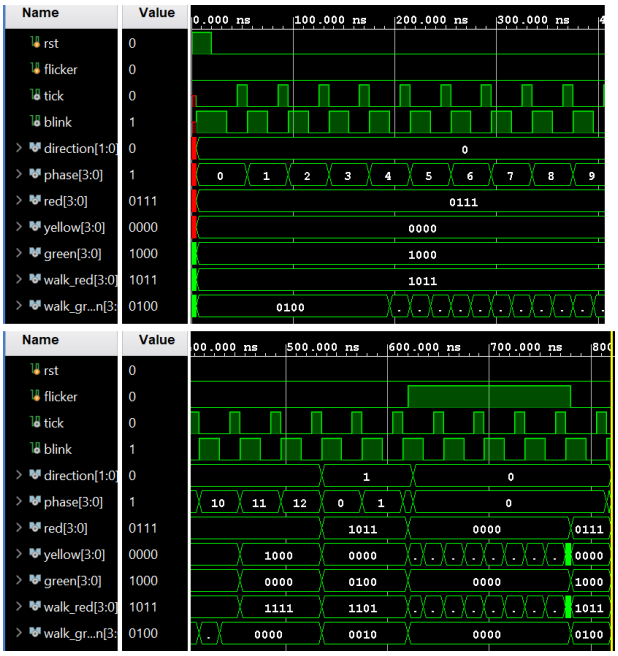


그림 4. FPGA-02 Vivado 파형(tb_view, CLK_HZ=4라 40 ns가 1초). 방향 0의 0~12단계, 방향 1의 시작, 점멸 모드.

보드 관찰

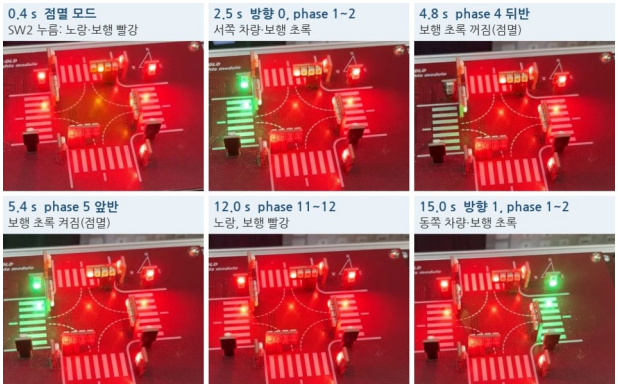


그림 5. FPGA-02 보드 사진. 입력 조건: 처음 약 1 s 동안 SW2(flicker)를 누르고 있다가 뗌. 확인한 동작: 점멸 모드 → 서쪽 초록 → 보행 점멸 → 노랑 → 동쪽 초록.

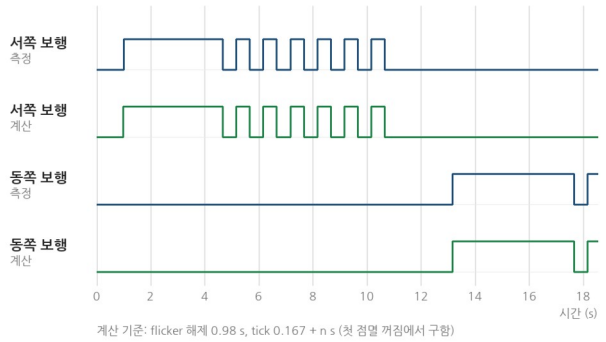


그림 6. FPGA-02 측정. 서쪽·동쪽 보행 초록의 측정(파랑)과 설계 규칙으로 계산한 값(초록). 계산의 기준은 첫 점멸 꺼짐 하나다.

시각 (s)	관찰	설계상 사건
0~0.98	노랑·보행 빨강 점멸	flicker=1
1.000	서쪽 차량·보행 초록	SW2 뗌, 방향 0 phase 0
4.667	서쪽 보행 초록 꺼짐	phase 4, blink 하강
5.167~10.667	0.5 s마다 켜짐·꺼짐	phase 5~10 점멸
11.167	서쪽 초록 모두 꺼짐	phase 11 노랑
13.167	동쪽 차량·보행 초록	방향 1 phase 0
17.667	동쪽 보행 초록 꺼짐	방향 1 phase 4

측정값:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/report/final/data/traffic_walk_green.csv

처음 0.98 s 동안은 SW2를 누르고 있어 점멸 모드였다. 0.17~0.67 s에 노랑과 보행 빨강이 켜지고 그 뒤 꺼져, blink가 0.5 s HIGH인 것과 맞는다. SW2를 떼자 서쪽 차량 신호와 서쪽 횡단보도가 바로 초록이 되었다. 보행 초록은 4.667 s에 처음 꺼지고 이후 0.5 s마다 켜지고 꺼지기를 여섯 번 반복했으며, 11.167 s에 차량 초록도 꺼지고 2 s 뒤 동쪽 방향이 초록이 되었다.

예상과의 비교

첫 점멸 꺼짐(4.667 s)을 「tick + 0.5 s」로 두면 tick 시각이 매초 x.167 s로 정해진다. 이 하나의 기준과 SW2를 뗀 시각만으로 계산한 나머지 보행 초록 변화 15번이 측정과 모두 같은 프레임이었다. phase가 1초 tick으로만 바뀌고, 방향 하나가 정확히 13초(1.000 → 13.167 s의 차이 12.167 s는 SW2를 뗀 뒤 첫 tick이 0.18 s 만에 와서다)인 것과 맞는다. 이 첫 tick이 1초보다 짧은 것은 second_tick이 flicker로 초기화되지 않기 때문으로, 예비보고서에서 예상한 그대로다.

버스 인덱스 3·2·1·0을 남·북·서·동으로 정한 XDC에서는 방향 0의 차량 신호가 남쪽(인덱스 3)이지만, 보드에서는 서쪽에 놓인 차량 신호와 서쪽 횡단보도가 함께 켜졌다. 모듈의 신호등 위치와 교재 핀 표의 방향 이름이 다르게 붙어 있는 것으로 보이며, 순서와 시간은 맞으므로 동작에는 영향이 없다. 노랑 램프는 주변의 빨강 빛에 묻혀 영상에서 색을 따로 구분하기 어려웠다.

5 FPGA-03 ADC

설계와 실습 중 수정

serial_frame16이 CS를 LOW로 둔 16 SCLK 동안 AD7908에 명령 16비트를 보내고 결과 16비트를 받는다. adc는 받은 워드의 하위 12비트를 sample에 저장하고 sample[11:4]를 LED1~8에 낸다. 채널은 DIP_SW1(add_sel[0])로 고른다.

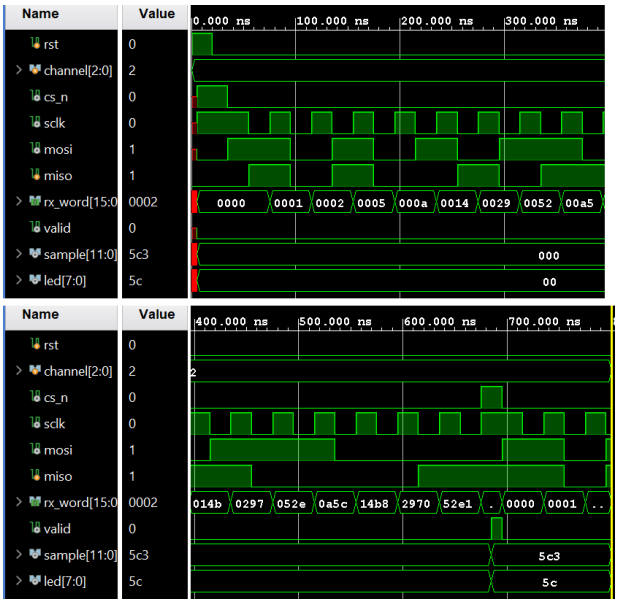


그림 7. FPGA-03 Vivado 파형(tb_view, HALF_CYCLES=2). mosi로 AB70을 내고 miso로 A5C3을 받아 sample=5c3, led=5c.

원본 bitstream에서는 DIP_SW1을 바꿔도 값이 바뀌지 않았다. AD7908은 DIN을 SCLK 하강 에지에서 읽는데 [4] 원본은 같은 하강 에지에 다음 비트를 내보내, 칩이 비트를 잘못 읽을 수 있었다. 실습 중에 다음 DIN 비트를 상승 에지에 내고 프레임 사이 CS HIGH 시간을 늘린 수정본(board_fix)을 만들어 보드에 올렸다. 영상은 이 수정본으로 찍었다.

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/lab5_03_adc/board_fix/serial_frame16.v

보드 관찰



그림 8. FPGA-03 보드 사진. 입력 조건: 0~4.4 s ch0(CdS) 위에 휴대전화를 댐, 4.4~5.2 s DIP_SW1로 ch1 선택, 5.6 s부터 주황색 가변저항(ADC IN1)을 돌림. 확인한 동작: LED1~8의 값이 입력에 따라 바뀜.

시각 (s)	ch	LED1~8	값	조작
0.5	0	1110_1110	238	휴대전화를 CdS 위에 댐
2.5	0	1110_0010	226	휴대전화를 CdS 위에 댐
3.0	0	1110_0010	226	휴대전화를 CdS 위에 댐
5.8	1	0000_0000	0	DIP_SW1 전환 직후 가변저항
6.5	1	0000_1010	10	가변저항을 돌림
7.0	1	0011_0000	48	가변저항을 돌림
9.0	1	1110_0110	230	가변저항을 돌림
12.0	1	1011_0110	182	가변저항을 돌림
17.0	1	1111_1110	254	가변저항을 돌림

측정값:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/report/final/data/adc_led_readout.csv

DIP_SW1을 바꾼 직후 LED가 모두 꺼졌고, 가변저항을 돌리는 동안 값이 0에서 1111_1110까지 크게 바뀌었다. CdS를 비추던 ch0의 값(1110_xxx0)과 다른 입력이 보이므로 채널 전환은 수정본에서 동작했다.

LED8이 켜지지 않은 원인

그런데 영상 전체(0.5 s 간격 36장면과 위 표)에서 최하위 LED8은 한 번도 켜지지 않았다. 가변저항을 돌리는 동안 최하위 비트가 매번 0일 확률은 매우 낮다. 또 최댓값이 1111_1111이 아니라 1111_1110이었고, 가변저항을 돌리는 동안 값이 0, 10, 48, 230, 182, 254로 크게 건너뛰었다.

AD7908은 CS 하강 에지에 첫 비트(선행 0)를 내고, 그 뒤 SCLK 하강 에지마다 다음 비트를 낸다[4]. 설계는 첫 SCLK 하강 다음의 상승 에지부터 받으므로 첫 비트를 놓치고 둘째 비트부터 받는다. 그러면 받은 워드가 한 칸 왼쪽으로 밀려, LED1~8에는 변환값의 아래 7비트와 뒤따르는 0이 나온다. 이 동작을 칩처럼 비트를 내는 모델(tb_dout.sv)로 시뮬레이션했다.

변환값	LED1~8 (시뮬레이션)	판정
0000_0000	0000_0000	같음
0000_0001	0000_0010	2배
0101_0101	1010_1010	2배
0111_1111	1111_1110	2배
1000_0000	0000_0000	MSB 잃음
1111_1111	1111_1110	MSB 잃음

로그:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/lab5_03_adc/board_fix/tb_dout_icarus.txt

시뮬레이션에서 LED8은 어떤 값에서도 0이고, 변환값이 128을 넘으면 MSB가 버려져 다시 0부터 보인다. 보드의 세 증상(LED8 항상 꺼짐, 최댓값 1111_1110, 돌리는 중 값이 크게 건너뜀)과 모두 맞는다. 다만 실제 DOUT 파형을 오실로스코프로 보지는 못했으므로, 이 설명은 영상과 시뮬레이션으로 뒷받침되는 원인 추정이다.

예상과의 비교와 수정 방향

예비보고서는 「입력 증가에 따라 LED 값이 이진수로 커짐」, 「최대 입력에서 모두 켜짐」을 예상했다. 채널 전환과 값의 변화는 나왔지만 최하위 비트와 최상위 비트가 맞지 않았다. 수신을 SCLK 하강 에지 직전에 하거나, 지금 구조를 유지한다면 LED에 rx_word[12:5]를 내면 한 칸 밀림이 상쇄된다. 예비보고서의 TB 모델은 첫 비트를 첫 SCLK 하강에 내도록 만들어서 이 문제를 잡지 못했다. 테스트벤치의 상대 모델을 설계자가 기대하는 대로가 아니라 데이터시트의 시간 그림대로 만들어야 한다는 점을 확인했다.

6 STM32_1 과제3 ISR로 LED 간격 변경

코드

버튼 ISR은 200 ms 잠금 검사 뒤 blink_ticks를 2배로(80 다음은 10) 바꾸고, 100 ms마다 오는 TIM6 ISR이 호출 횟수를 세어 blink_ticks에 이르면 LD2(PA5)를 토글한다. 간격을 바꿀 때 count는 그대로 두므로 새 간격은 직전 LED 변화부터 센다[1][3].

```
static void button_isr_p7(void)
{
  static uint32_t last_ms = 0;
  uint32_t now_ms = HAL_GetTick();
  if (now_ms - last_ms < BUTTON_LOCK_MS)
  {
    return; /* 직전 누름 뒤 200 ms 안의 호출은 접점 떨림 */
  }
  last_ms = now_ms;
#if STM32_1_TASK == 3
  blink_ticks = (blink_ticks >= 80U) ? 10U : blink_ticks * 2U;
#else
  pwm_compare = (pwm_compare >= 750U)
        ? 250U : pwm_compare + 250U;
  __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_compare);
#endif
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/stm32_lab/Core/Src/ece2_474_routine.c#L87-L103

```
static void tim6_isr_p4(void)
{
#if STM32_1_TASK == 3
  static uint32_t count = 0;
  if (++count >= blink_ticks)
  {
    count = 0;
    HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
  }
#endif
}
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/stm32_lab/Core/Src/ece2_474_routine.c#L109-L119

보드 관찰

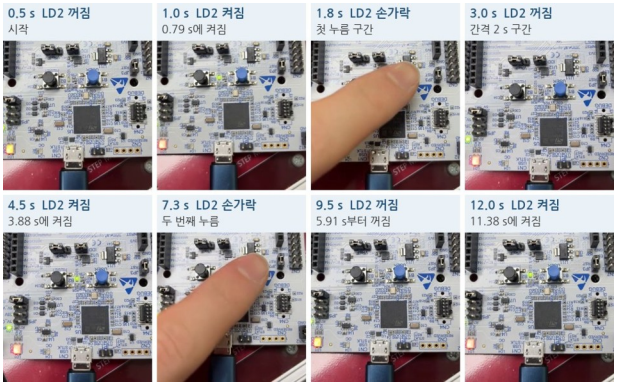


그림 9. 과제3 보드 사진. 입력 조건: 디버그 모드로 실행, 사용자 버튼(파란색)을 1.3~2.2 s, 6.9~7.7 s에 누름. 확인한 동작: 검은 리셋 버튼과 파란 버튼 사이의 LD2가 점점 긴 간격으로 바뀐다. 아래 왼쪽의 LD1(COM)이 적·녹으로 깜빡인다.

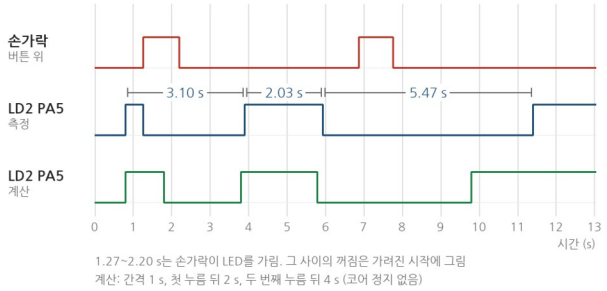


그림 10. 과제3 측정. 가운데가 영상에서 읽은 LD2, 아래가 첫 변화(0.80 s)와 코드의 규칙으로 계산한 값이다.

측정 (s)	사건	간격	계산 (s)	차이
0.800	LD2 켜짐	—	기준	—
1.27~2.20	손가락, 그 사이 꺼짐·누름	—	1.80 꺼짐	—
3.900	LD2 켜짐	3.10 s	3.80	+0.10
5.933	LD2 꺼짐	2.03 s	5.80	+0.13
6.87~7.77	손가락, 두 번째 누름	—	—	—
11.400	LD2 켜짐	5.47 s	9.80	+1.60

측정값:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/report/final/data/stm32_task3_led.csv

누르기 전에는 1초 간격이었고, 첫 누름 뒤의 두 변화 사이(3.90 → 5.93 s)는 2.03 s, 두 번째 누름 뒤에는 5.47 s였다. 간격이 누를 때마다 늘어나는 순서는 맞지만, 계산과 비교하면 첫 누름 뒤 0.10 s, 두 번째 누름 뒤 1.47 s가 더 걸렸다. 표의 차이는 누적값이라, 두 번째 누름 뒤 늘어난 몫은 1.60 − 0.13 = 1.47 s다. 2.03 s 구간은 손가락이 없는 동안이므로 한 프레임 이내로 2 s와 같다.

간격이 길어진 원인: 디버그 모드

이 영상은 디버그 모드로 내려받아 실행한 상태에서 찍었고, ST-LINK의 LD1(COM)이 적·녹으로 번갈아 깜빡여 디버거와 통신 중인 것이 보인다. 코어가 디버거에 의해 멈추면 그동안 TIM6 ISR이 실행되지 못하므로, 멈춘 시간만큼 count가 늘지 않고 LED 변화가 늦어진다. LAB4 과제1에서 중단점 때문에 누를 때마다 약 2.8 s씩 늦어진 것과 같은 현상이다.

두 번째 누름의 1.47 s는 버튼 ISR 안에서 멈췄다가 계속 실행하기까지의 시간으로 자연스럽다. 첫 누름의 0.10 s는 TIM6 호출 한 번이 빠진 크기인데, 사람이 중단점을 보고 계속 실행을 누르기에는 짧다. 측정 오차(±0.033 s)보다는 크므로 0.10 s의 원인은 이번 자료로 정하지 못했다. 두 번의 누름이 각각 한 단계만 바꾼 점(2 s, 그다음 4 s)은 디버그 정지와 무관하게 확인된다. 특히 두 번째 누름은 손가락이 0.9 s 동안 버튼 위에 있었고 클릭음이 네 번 났지만 8 s로 넘어가지 않았다.

예상과의 비교

1 → 2 → 4 s 순서, 새 간격이 직전 LED 변화부터 세어지는 것, 길게 눌러도 한 단계만 바뀌는 것은 예상과 같다. 8 s와 8 → 1 s 순환은 영상이 13 s로 짧아 나오지 않았다. 예비보고서에서 걱정한 「뗄 때의 떨림으로 한 단계 더 넘어감」은 두 누름에서 나타나지 않았다.

7 STM32_1 과제4 버튼으로 PWM 비율 변경

코드

TIM3은 100 MHz를 100분주한 1 MHz로 0~999를 세어 주기 1 ms의 PWM을 PB4에 낸다. 버튼 ISR이 비교값 pwm_compare를 250 → 500 → 750 → 250으로 바꿔 CCR1에 쓰므로 듀티비는 25 → 50 → 75 → 25 %로 순환한다[3].

```
/* button_isr_p7(), STM32_1_TASK == 4 */
pwm_compare = (pwm_compare >= 750U)
        ? 250U : pwm_compare + 250U;
__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_compare);
```

GitHub:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/stm32_lab/Core/Src/ece2_474_routine.c#L99-L101

오실로스코프 관찰

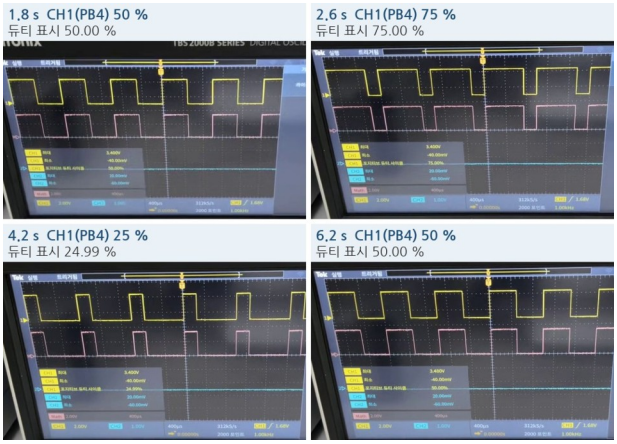


그림 11. 과제4 오실로스코프 화면. 입력 조건: CH1(노란색)에 PB4, 400 µs/div, 화면 밖에서 버튼을 누름. 확인한 동작: 주기는 그대로이고 HIGH 폭만 바뀐다. 분홍색(Math) 파형은 측정에 쓰지 않았다.

영상 구간 (s)	듀티 표시 (%)	예상
1.6~2.0	50.00	50 % (CCR1 500)
2.2~3.6	74.90~75.24	75 % (750)
3.8~5.0	24.99~25.00	25 % (250)
5.4~6.4	49.83~50.00	50 % (500)
6.6~6.8	74.91~74.99	75 % (750)

측정값:
https://github.com/tae2yi/UOS_ece2/blob/4ff988d/lab5/report/final/data/pwm_duty_readout.csv

화면의 측정 표시는 영상 내내 최대 3.40~3.48 V, 최소 −40 mV, 트리거 주파수 1.00 kHz였다. 듀티비는 버튼을 누를 때마다 한 단계씩 바뀌었고 75 % 다음에 25 %로 돌아갔다. 영상은 이미 몇 번 누른 뒤인 50 %에서 시작한다.

예상과의 비교

주기 1 ms(1 kHz)와 25 % 단위의 순환, 75 → 25 %의 되돌림이 예상과 같다. 측정 듀티비의 차이는 최대 0.24 %p로, 400 µs/div에서 화면 2000점(한 점 2 µs)으로 HIGH 폭을 재므로 한 주기에서 1~2점 차이에 해당한다. 최대 전압이 3.3 V보다 약간 높게 나온 것은 프로브·화면 측정의 오차 범위로 본다. 주기 중간에 비교값을 바꿔도 파형이 잘리지 않는지(미리 읽기)는 영상의 시간 분해능으로는 볼 수 없었다. 디버거 Watch 화면은 이번 기록에 없다.

8 토론

예상과 실제의 비교

예상 (예비보고서)	실제	판정
버튼 1~8: 도~높은 도	8음, 설계값 ±0.11 Hz	일치
버튼 뗌: 무음	음 사이 무음	일치
신호등 0~3 초록, 4~10 점멸	4.667 s부터 0.5 s 점멸	일치
11 s 노랑, 13 s 다음 방향	11.167, 13.167 s	일치
flicker: 노랑·보행 빨강 점멸	처음 0.98 s	일치
ADC 채널 변경	수정본에서 전환	일치
ADC 최대 입력에서 모두 켜짐	최대 1111_1110	불일치
과제3 1 → 2 → 4 s	2.03 s, 그다음 4 s+정지	규칙 일치
길게 눌러도 한 단계	0.9 s 누름에 한 단계	일치
과제4 25 → 50 → 75 → 25 %	50 → 75 → 25 → 50 → 75 %	일치
PWM 주기 1 ms	1.00 kHz	일치
8 s 간격, 디버거 Watch	영상에 없음	미확인

카운터로 만드는 시간

이번 주의 다섯 실험은 모두 빠른 기준 신호를 세어 느린 동작을 만든다. 피아노는 50 MHz를 반주기 클록 수만큼 세어 262~523 Hz를, 신호등은 50,000,000을 세어 1초 tick을, 과제3은 100 ms TIM6 호출을 10·20·40번 세어 1·2·4초를, 과제4는 1 MHz를 1000까지 세어 1 ms 주기를 만든다. 피아노와 신호등은 FPGA 클록이 멈추지 않아 측정값이 계산과 0.1 Hz, 0 프레임까지 맞았고, 과제4의 PWM도 타이머 하드웨어가 만들어 정확했다. 반면 과제3은 횟수를 ISR이 세므로 코어가 멈추면 시간도 함께 멈췄다.

오차와 한계

- 영상의 시간 분해능은 0.033 s다. 신호등·과제3의 비교는 이 범위 안에서만 의미가 있다.
- ADC LED 값과 듀티 표시는 사람이 장면을 읽은 값이다. LED 하나하나는 확대 장면에서 분명했지만 값이 빠르게 바뀌는 순간은 고르지 않았다.
- ADC 원인은 시뮬레이션으로 재현했지만 DOUT 파형을 직접 측정하지 않았고, 고친 bitstream을 보드에서 시험하지 못했다.
- 과제3 첫 누름의 0.10 s 지연의 원인은 정하지 못했다.

개선 방법

- ADC: LED에 rx_word[12:5]를 내는 bitstream으로 최댓값 1111_1111과 LED8 변화를 확인하고, 오실로스코프로 CS·SCLK·DOUT을 함께 본다.
- 과제3: 중단점 없이(또는 디버거 없이) 리셋부터 1 → 2 → 4 → 8 → 1 s 한 바퀴를 다시 찍는다.
- 과제4: 디버거 Watch 화면과 버튼 누름이 함께 보이도록 오실로스코프와 보드를 한 화면에 담는다.

9 결론

FPGA 피아노는 여덟 음의 주파수가 설계값과 0.11 Hz 이내였고, 신호등은 SW2 점멸 모드에서 시작해 초록 → 보행 점멸 → 노랑 → 다음 방향의 모든 변화가 설계 규칙의 계산과 같은 프레임에 일어났다. ADC는 DIN 출력 시점을 고친 수정본에서 채널 전환이 되었으나, LED8이 한 번도 켜지지 않았다. AD7908의 출력 시점대로 만든 모델로 시뮬레이션해 수신이 한 비트 늦어 값이 2배로 보이고 MSB를 잃는다는 것을 재현했다.

STM32_1 과제4는 버튼을 누를 때마다 PB4의 듀티비가 25 % 단위로 순환하고 주기 1 kHz가 유지되었다. 과제 3은 LED 간격이 1 → 2 → 4 s로 늘고 길게 눌러도 한 단계만 바뀌었지만, 디버그 모드에서 코어가 멈춘 시간만큼 변화가 늦어졌다. 하드웨어 카운터로 만든 시간은 정확했고, ISR이 세는 시간은 코어 정지에 영향을 받는다는 차이가 측정값에 그대로 나타났다.

10 제출 자료

항목	위치
저장소	https://github.com/tae2yi/UOS_ece2
커밋	4ff988dea610ddbb3bcd9c3ec4c2c77b89afe200
MCU 코드	lab5/stm32_lab/Core/Src/ece2_474_routine.c
FPGA 설계	lab5/lab5_01_piano, lab5_02_traffic, lab5_03_adc
ADC 수정본	lab5/lab5_03_adc/board_fix
영상·설명	lab5/report/final/videos/README.md
측정값	lab5/report/final/data/*.csv

소스: https://github.com/tae2yi/UOS_ece2/tree/4ff988d/lab5

영상:
https://github.com/tae2yi/UOS_ece2/tree/master/lab5/report/final/videos

참고 문헌

[1] Joseph Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed., Newnes, 2013. ISBN 978-0-12-408082-9.

[2] M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog, 6th ed., Pearson, 2018. ISBN 978-0-13-454989-7.

[3] STMicroelectronics, STM32CubeG4 MCU Firmware Package (stm32g4xx_hal_tim.c, stm32g4xx_hal_gpio.c). https://github.com/STMicroelectronics/STM32CubeG4

[4] Analog Devices, AD7908/AD7918/AD7928 8-Channel, 1 MSPS, 8-/10-/12-Bit ADCs with Sequencer, Data Sheet.